Question 1 Answer: In C#, structs are value types and do not support inheritance from other structs or classes (only from System.ValueType) to keep them lightweight and avoid the complexity of inheritance hierarchies.

Question 2 Answer: Access modifiers define where a class member can be accessed — for example, private limits access to within the class, internal allows access within the same assembly, and public allows access from anywhere.

Question 3 Answer: Encapsulation hides internal data and implementation details, exposing only what's necessary, which improves security, reduces errors, and makes code easier to maintain.

Question 4 Answer: Constructors in structs initialize their fields when an instance is created, ensuring all fields are assigned before use — they can be parameterized but not explicitly default-defined.

Question 5 Answer: Overriding methods like ToString() provides a custom, human-readable string representation of an object, making debugging and logging more intuitive.

Question 6 Answer: Structs (value types) are allocated on the stack and copied when passed around, while classes (reference types) are allocated on the heap and accessed via references.